QueryBook — Drawings and Description of the Drawings

FIG. 1–57 · companion to the Provisional Patent Application (synchronized, with support for agent-wrapper services and the self-knowledge layer)

October 22, 2026

Brief Description of the Drawings

The accompanying drawings, incorporated in and forming part of the specification, illustrate embodiments of the invention and, together with the description, serve to explain its principles. In the drawings, like reference numerals designate corresponding parts throughout the several views.

Platform (FIG. 1–18)

FIG. 1 is a block diagram of the system, showing the ingestion pipeline, the storage engine, the UFCS substrate, the FQL query engine, the verification ledger, and the federation boundary.

FIG. 2 is a diagram of a Fact Atom carried within a Fact Envelope, showing the nucleus, group classification, polarity, and links of the atom, together with the envelope's confidence, provenance, certification, and content-derived identifier.

FIG. 3 is a diagram of a fact-certification authority hierarchy, showing a root trust anchor, domain authorities, and the certified facts and trust scores derived beneath them.

FIG. 4 is a diagram of the substrate relationships, showing a pairwise graph, a hyperedge, and a directed hyperchain among Fact Units.

FIG. 5 is a diagram supporting claim 11, showing tour/exhibition sequencing of immersive scenes into ordered waypoints.

FIG. 6 is a diagram of a query that filters jointly on fact structure and certification trust, and of the trust-native execution by propagation, branch-and-bound pruning, and contradiction exclusion.

FIG. 7 is a flowchart of verifying machine-generated text against certified records, classifying each assertion as verified, contradicted, or unknown.

FIG. 8 is a diagram of federation, showing a certified envelope crossing a boundary and being re-validated on the receiving side.

FIG. 9 is a diagram of fact-level erasure using an externally held key (cryptographic shredding of a sealed payload).

FIG. 10 is a diagram of the Universal Security Architecture, showing the twelve defensive layers, the five simultaneous defensive postures, and key mechanisms including the AI Attack Prediction Engine.

FIG. 11 is a diagram of Universal Honeypot Saturation and the Data Firehose, showing port saturation via the Phantom Port Daemon and application-entry-point saturation via the Shadow-App mechanism.

FIG. 12 is a diagram of the five deployment topologies — cloud SaaS, local server, hybrid, single-device, and edge — over the invariant common layer of safety, governance, and provenance.

FIG. 13 is a diagram of the Adaptive Response Engine, showing the five-stage pipeline from Result-Set Analysis through Generation-Directive Assembly, with the directive fixed before generation begins.

FIG. 14 is a diagram of the user-owned agent system, showing Blueprint definition and registration, Agent-Instance creation under commissioner-bounded authority, and the governance evaluation at each action.

FIG. 15 is a diagram of deterministic multi-language regeneration, showing source ingestion to Fact Units, the language-independent Fact Unit graph as a single knowledge source, and regeneration into any target language.

FIG. 16 is a diagram of semantic compression, expansion, colorization, and resolution upscaling, showing the meaning-storage compression flow and the on-demand regeneration of a surface representation.

FIG. 17 is a diagram of the QueryBook-enhanced TCP/IP hybrid protocol packet, showing the optional semantic-metadata header carrying FUID, ProvenanceRef, and confidence fields alongside the standard headers and payload.

FIG. 18 is a diagram of the Neuromorphic Cognitive Layer and attractor network, showing the energy function, the capacity ceiling, and convergence to a stable equilibrium.

Confidence, packet, framing and query (FIG. 19–22)

FIG. 19 is a diagram of the Fact Unit confidence and evidence model, comprising an evidence pair, a Beta posterior, a score, a credible interval, source diversity, temporal decay, and a confidence trend.

FIG. 20 is a diagram of the UFCS packet structure, showing the fields of a serialized Fact Unit record and an exemplar packet.

FIG. 21 is a diagram of UFCS packet framing, in which immutable packets are de-duplicated by fingerprint and appended to content-addressed blocks under a fingerprint index.

FIG. 22 is a diagram of FQL query anatomy with a worked example, in which match, where, exclude, rank, and return clauses produce a highest-trust answer bound to its evidence by a provenance hash.

Language and speech subsystem (FIG. 23–30)

FIG. 23 is a block diagram of the overall QueryBook Language Expression Layer (LEL) system architecture.

FIG. 24 is a block diagram of the Sub-Language Priming Layer (SLPL) and the measurable representation produced by each of its six components.

FIG. 25 is a flow diagram of the developmental learning sequence and the one-way Transition Gate having four readiness thresholds.

FIG. 26 is a block diagram of the multilingual Delta Acquisition Model and the acquisition acceleration flywheel.

FIG. 27 is a block diagram of the seven-stage Speech Output Pipeline.

FIG. 28 is a data-flow diagram of the co-grounded integration of the LEL with the Federated Query Language (FQL) and the Universal Federated Connectivity System (UFCS).

FIG. 29 is a block diagram of a reduced-to-practice embodiment (the Language Lab) implementing Phase One, the Transition Gate, and a deterministic Phase Two.

FIG. 30 is a data-flow diagram of the deterministic Phase-Two semantic grounding of the reduced-to-practice embodiment, in which word meanings are attached only from auditable sources — a bundled public-domain dictionary and self-grounding against verified Fact Units in the store — with any large-language-model meaning source excluded from the grounding path.

Further subsystems and embodiments (FIG. 31–49)

FIG. 31 is a block diagram of the advanced multilingual pronunciation subsystem and self-contained speech output of the reduced-to-practice embodiment, showing the pronunciation preference ladder (bundled open-source phonemizer, then per-language rule-seeded grapheme-to-phoneme tables, then an explicit non-fabricated null), the single-call phrase transcription that keeps the displayed transcription consistent with the synthesized audio, and the self-contained bundle and engine-selection ladder that produce offline audio or an honest refusal without fabricating audio.

FIG. 32 is a flow diagram of the permanent, deterministic pronunciation-and-speech quality-control battery of the reduced-to-practice embodiment, which, for each supported language and without a large language model, verifies that a phonemic transcription is produced, that the syllable count is within tolerance, and that a short phrase synthesizes to a valid audio waveform, returning a per-language report and an overall pass score.

FIG. 33 is a data-flow diagram of the Language & Identity Linking (LIL) voiceprint enrollment and matching subsystem, in which a consent-gated capture produces an acoustic speaker embedding (voiceprint), a match against stored voiceprints is passed through a QueryBook trust gate returning VERIFIED, CONTRADICTED, or UNKNOWN, and verified identity links are written as provenance-tracked, trust-scored Fact Units in the identity graph, with cross-script name and entity resolution performed by the language subsystem.

FIG. 34 is a flow diagram of the LIL trust-gated voice authentication process, in which a claimed-identity verification produces a match score and anti-spoofing signals that feed a deterministic risk-scoring policy and a decision gate returning accept, challenge (second factor), or reject, each decision recorded as a provenance-tracked auth-event Fact Unit.

FIG. 35 is a block diagram of the Universal Integrity Audit Service (UIAS), in which connector and ingestion agents observe an unmodified monitored system at multiple workflow positions and

seal each observable event as a content-addressed Fact Unit carrying a content hash, a one-way submitter-identity hash, an independent nanosecond timestamp, a source fingerprint, and the sealing agent's identity; a similarity-detection agent sets a contemporaneous cross-party anomaly flag over content signatures without storing content; and a verification agent continuously re-verifies the append-only Fact-Unit ledger.

FIG. 36 is a flow diagram of the UIAS signed Proof Package generation, in which, on an accusation or inquiry, a proof-package agent assembles verified sealed Fact Units, the similarity-log entry, a chronological ledger extract bound by the response-provenance hash, and a factual determination from the arbitration agent under the Prime-Directive gate, and signs the package under the UIAS independent identity to produce independent, pre-dispute, tamper-proof, complete, and admissible evidence.

FIG. 37 is a data-flow diagram of the Paralinguistic & Pragmatic Interpretation Layer (PIL) applied to an ingested voice, in which a paralinguistic analyzer produces measured acoustic readings and a labeled arousal estimate while a pragmatic framer applies sourced context, perspective, ergonomics, culture, and social-mores frames, the two combining into an Interpretation Frame written as provenance-tracked Fact Units, with estimates labeled and never asserted as true feelings.

FIG. 38 is a flow diagram of the PIL human-query interpretation, in which a query is reduced to an Interpretation Frame, intent is disambiguated from context and perspective, an ambiguity gate asks one clarifying question when intent is materially uncertain, a register-and-ergonomic-mode selector sets the reply style and delivery from sourced culture/mores/ergonomics packs, and the existing retrieve-gate-compose pipeline answers from verified Fact Units in that register without altering the facts.

FIG. 39 is a block diagram of the Lecture Query mode ingestion pipeline, in which a live or recorded audio/video program is captured under consent, transcribed and diarized, analyzed by the paralinguistic layer over time and by the language layer for structure and claims, summarized deterministically, and ingested as a Lecture Record whose claims are stored as attributed utterance facts and which is forever-linked to the speaker, venue, event, organization, and date by immutable provenance-tracked, trust-gated identity links.

FIG. 40 is a data-flow diagram of the forever-linked Lecture Record in the Fact-Unit identity graph and its querying, showing the record bound to speaker, venue, event, and organization entities and to the transcript and paralinguistic track, its attributed claims gated as corroborated, contradicted, or open, and a query answered with citations to the record and attribution to the speaker, refused where unknown.

FIG. 41 is a data-flow diagram of deterministic multilingual acquisition without a generative model, in which a per-language learning agent acquires vocabulary, phonology (IPA), and grammatical regularities from a lawful public-domain corpus and a governed bilingual dictionary, an IPA transcription is verified against a reference phonemizer and withheld rather than asserted when verification fails, and each accepted item is written as a provenance-tracked lexical record

reproducible from the same inputs, with no large or generative language model in the grounding path.

FIG. 42 is a block diagram of a Dialect Parameter Cluster (DPC), a declared and versioned cluster of phonological, lexical, and orthographic parameters by which an input utterance is attributed to a dialect by a deterministic parameter-match score, the cluster modulating expression only and never altering admission, applicability scope, confidence, or the governed vocabulary.

FIG. 43 is a data-flow diagram of grounded scene compilation to an external generative media engine, in which a grounded scene representation derived from verified Fact Units is compiled and validated into a prompt bundle for an external generative video engine and any returned media is admitted only as a derived artifact marked as a reconstruction and never as a grounding source.

FIG. 44 is a block diagram of key-safe external-service mediation, in which a credential for an external service is held on the server side of the query/presentation boundary and never transmitted to the client, the server performing the external call and serving the result to the client by an opaque cache identifier so that the credential never crosses the boundary.

FIG. 45 is a flow diagram of scene reconstruction from prose, in which a deterministic reconstructor derives a shot sequence, a timed prompt sequence, and pacing and music cues as a bounded reconstruction, renders a participant who is not an identified subject as a silhouette so that no unverified identity is fabricated, and marks the whole as a reconstruction.

FIG. 46 is a block diagram of modular edition packaging, in which the system is deployable as one of a plurality of nested capability editions selected at launch, each edition enabling a declared subset of subsystems while the Prime Directive and the deterministic reasoning core remain invariant across every edition.

FIG. 47 is a block diagram of the version-independent, provenance-continuous store, in which the knowledge store resides at a canonical location independent of the deployment software version so that successive software versions bind to the same provenance-tracked records, a version change never re-initializes the store, and the system refuses to substitute an empty store where a populated store exists.

FIG. 48 is a data-flow diagram of a cited question-and-answer (ChatBot) application embodiment, in which a user question is interpreted for intent and register, answered by Federated Query Language retrieval over verified Fact Units, passed through the Prime-Directive gate, and returned either as a composed answer carrying per-fact citations or, where the evidence is insufficient, as an honest UNKNOWN, the same citation-or-UNKNOWN contract holding when the embodiment is deployed as a ChatBot or assistant surface.

FIG. 49 is a block diagram of the application and deployment surfaces of the system over its invariant core, in which a single governed body of Fact Units is presented through an on-device micro-encapsulated eBook or document question-and-answer surface, a software-as-a-service or application-programming-interface service, a Fact-Unit compressed search engine, a deterministic large-language-model-hybrid adjunct, and a private or offline analysis deployment, each surface

inheriting the same citation-or-UNKNOWN contract and provenance and none able to assert a fact the core would refuse.

Agent-era extensions (FIG. 50–55)

FIG. 50 is a block diagram of Open Claw ingestion, showing content-based format detection, archive bounds, the Artifact Segmenter, the instruction-like content screen, the OpenClaw Interpreter, and the admission of artifact-description, structured, passage, and flagged untrusted records to the Fact Unit store under a provenance seal.

FIG. 51 is a block diagram of on-chain certification, showing a QueryBook object and its local provenance seal, the content hash and on-chain metadata, the in-system transaction signer and network guard, the certificate contract on a public blockchain, the independent verifier, and the licence check.

FIG. 52 is a block diagram of the agent protection and deception layer, showing the HTTP gate, the isolation register, the Agent Gateway, the deception check with honeycode, honeytokens, tripwires, and honeypot agents, the AI-aware firewall and anomaly detector, the security event bus, the decoy environment, the engagement recorder, and operator-only release.

FIG. 53 is a block diagram of the tamper-evident security record, showing append-only entries, the hash chain, write-once signed Merkle checkpoints, a mirror on a second device, a blockchain anchor, the recompute-from-content verifier with inclusion proofs, and the assurance report.

FIG. 54 is a block diagram of agent interoperability, showing an external AI agent and its skill definition, the Model Context Protocol tool server, the tool-to-command mapping, the Agent Gateway with the agent’s allow-list, grounded handlers, and isolation upon a reserved command.

FIG. 55 is a block diagram of the framed hybrid transport, extending FIG. 17, showing per-type compression, the frame with its header, metadata, payload, CRC32 checksum, and optional signature, the control, normal, and bulk lanes, the receiver, package rebuild with manifest check, and record verification into the Fact Unit store.

Support for agent-wrapper services (FIG. 56)

FIG. 56 is a block diagram of the support interfaces QueryBook provides to external agent-wrapper services such as Verified Agent Transport, grouped as verification, transport, record and certificate, and governance interfaces over the Fact Unit store, security log, certificates, Agent Gateway and MCP server, and framed hybrid transport.

Self-knowledge (FIG. 57)

FIG. 57 is a block diagram of the self-knowledge layer, showing a self-question resolved through a specification index, an evidence-checked implementation map over the program's own source files and self-test results, a status labeller, an aspect describer, a consistency checker and gap agenda, a rule-based answerer with a disclosure filter, and the interfaces through which the answer, with its label and sources, is returned.

Detailed Description of FIG. 56

FIG. 56 shows the interfaces QueryBook provides to external agent-wrapper services — Verified Agent Transport (VAT) and similar relays that sit between an AI agent and the network and check what the agent says and does. The wrapper service itself is outside the system and is described separately; FIG. 56 shows only what QueryBook offers it.

The wrapper service (5690) receives the traffic of an AI agent (5695) bound for a destination (5697) and calls the support interfaces (5600) under an identity registered with the Agent Gateway. Verification is read-only and records are append-only: a wrapper service never writes Fact Units.

Reference numerals

Numeral	Element	Rests on
5600	QueryBook support interfaces	
5610	Verification interfaces	FIG. 6, 7, 21, 22
5612	Claim verification endpoint (read-only): candidate Fact Units or fingerprints in; verified, contradicted or unknown out, with citation and trust	FIG. 22
5614	Contradiction query template: same subject and predicate, conflicting value or opposite polarity, trust at or above the minimum	FIG. 6, 7
5616	Fingerprint-only verification mode: fingerprints and canonical subject/predicate identifiers only	FIG. 21
5618	Triple extraction and assertion-type labels (fact, opinion, plan, prediction)	FIG. 2, 6
5620	Transport interfaces	FIG. 55
5622	Frame message types 16–31 reserved for registered extensions; unregistered types dropped and recorded	FIG. 55
5624	Extension flag (0x0040) and reserved header field as 16-bit session number and 16-bit hop count	FIG. 55
5626	Python frame codec; control, normal and bulk lanes as QUIC streams with TLS-on-TCP fallback	FIG. 55
5628	Have/want fingerprint reconciliation; Bloom filter above 4,096 fingerprints	FIG. 21, 55
5630	Record and certificate interfaces	FIG. 51, 53
5632	Signed verdict records and published verifier key	FIG. 53
5634	Append by a registered service to the security log	FIG. 53
5636	Session-range Merkle root and inclusion proofs	FIG. 53
5638	Third-party session certificate type, sealed as a Fact Unit	FIG. 51
5640	Governance interfaces	FIG. 52, 54
5642	Wrapper service registration (read-only verification, append-only record rights)	FIG. 52

Numeral	Element	Rests on
5644	Signed, versioned, pull-only policy bundle export	FIG. 52
5646	Tool-server pre-call and post-call hooks; hash of each tool result record	FIG. 54
5648	Principal evidence pair (agent reputation); restriction only on a recorded operator decision	FIG. 19, 52
5650	Fact Unit store and FQL	FIG. 2, 6, 21, 22
5660	Security log	FIG. 53
5670	Certificates	FIG. 51
5680	Agent Gateway and MCP server	FIG. 52, 54
5685	Framed hybrid transport	FIG. 55
5690	External agent-wrapper service (e.g., Verified Agent Transport) — outside the system	
5695	AI agent	
5697	Destination (agent, tool or data source)	

Status

The support interfaces of FIG. 56 are specified and not yet implemented; no performance figures are asserted for them. They correspond to features 364–382 of the QueryBook Composite Feature Registry v68 and to claims 355–357 of the application.

Detailed Description of FIG. 57

The Self-Knowledge Layer (SKL) is the part of QueryBook that describes QueryBook. Asked what it is, how it reasons, whether it is neuromorphic, whether it predicts, or whether it improves itself, the system answers from three verified layers and from nothing else: the specification (the numbered sections of this Overview), the implementation (the module and symbol that carry out each section, checked against the program's own source files at the moment of asking), and the evidence (the host application's own self-tests, diagnostics and measured scores). The layer never generates its self-description with a language model, and it never repeats a claim its own code does not support.

A self-question (5700) from a user, the assistant or an application is received by the self-knowledge layer (5705). A specification index (5710) holds the numbered sections of the specification and resolves the question to the sections that specify the capability asked about. An implementation map (5720) holds, for each section and each reasoning aspect, evidence pairs consisting of a source file and a symbol that must appear in it; the pairs are checked against the program's own source files (5795) at the moment of use, the files being read as text and not executed, and the host application's self-test and measurement results (5797) are attached as further evidence. A status labeller (5730) assigns exactly one of the labels built and tested, partly built, specified and not yet built, not planned by design, and not in this application; a capability is labelled built only when all of its evidence is present. An aspect describer (5740) describes reasoning aspects — deterministic reasoning, atomic Fact Units, world model, predictive cognition, neuromorphic layer, hypothesis generation, simulation, deduction, feedback loops, supervised self-improvement, recursive self-improvement and self-knowledge — at selectable levels of detail. A consistency checker (5750) reports as a contradiction any claim labelled built whose evidence is missing, and any capability labelled specified whose implementation names are found in the program; a contradicted claim is withheld. A gap agenda (5760) lists the specified and partly built sections with their nearest built components. A rule-based answerer (5770) composes the answer with its label, section numbers and file-and-symbol sources, and a disclosure filter (5780) removes anything resembling a key or credential. Interfaces (5790) expose the summary and single answers to applications, a self-knowledge page and an administrative panel. No language model is used: the same question about the same program gives the same answer.

Reference numerals

Numeral	Element	Rests on
5700	Self-question (user, assistant or application)	FIG. 57
5705	Self-knowledge layer	FIG. 57
5710	Specification index: numbered sections §1–§56	FIG. 57
5720	Evidence-checked implementation map: (file, symbol) pairs	FIG. 57
5730	Status labeller: built / partly built / specified / not planned / not present	FIG. 57
5740	Reasoning-aspect describer: twelve aspects, three levels	FIG. 57
5750	Self-consistency checker: contradictions withheld	FIG. 57

Numeral	Element	Rests on
5760	Gap agenda: specified and partly built sections	FIG. 57
5770	Rule-based answerer: label, sections, sources	FIG. 57
5780	Disclosure filter: keys and credentials removed	FIG. 57
5790	Interfaces: /api/self, /api/self/ask, page, panel, assistant	FIG. 57
5795	The program's own source files (read as text)	FIG. 57
5797	Self-tests and measurements of the host application	FIG. 57
5799	Answer with status label and sources	FIG. 57

Status

The self-knowledge layer of FIG. 57 is built and tested in prototype v9.64 (qb_selfknow.py) and in the shared core of the applications (qbselfknow.py). It corresponds to features 383–391 of the QueryBook Composite Feature Registry v70, to Section 56 of the Core Technology Overview v15 and to claims 358–362 of the application.